University of Economics, Prague

Faculty of Informatics and Statistics

DDD INFRASTRUCTURE LIBRARY ON .NET CORE PLATFORM USING MONGODB

BACHELOR THESIS

Study program: Applied informatics

Field of study: Applied informatics

Author: Maxim Dužij

Supervisor: RNDr. Helena Palovská, Ph.D.

Prague, May 2020

Declaration

I hereby declare that I am the sole author of the thesis entitled “DDD Infrastructure library on .NET Core platform using MongoDB. “ I duly marked out all quotations. The used literature and sources are stated in the attached list of references.

In Prague on ……. Signature

Maxim Dužij

Acknowledgement

I also place on record my sense of gratitude to one and all who, directly or indirectly, have lent their helping hand in this venture. I would especially like to express my appreciation and gratitude to the supervisor of my thesis, RNDr. Helena Palovská, Ph.D.

Abstract

DDD (Domain-Driven Design) is a popular approach to complex software development. However, there are many ways to implement such an approach to the .NET Core platform using the MongoDB database.

The main goal of this bachelor thesis is the creation of a framework that could help me, as a developer, speed up early development and use the DDD approach in my .NET application. The Framework will include not only common DDD objects and interfaces (typed identifiers, domain aggregates, and domain events) but also include the persistence layer where it will use MongoDB database engine.

Part of this bachelor's work will also include sample Library domain and Library client application (Razor pages) what will use the framework and demonstrate basic principles of DDD.

Keywords

Domain-Driven Design, DDD, MongoDB, .NET Core

Content

Introduction 8
1 Development Process Plan 10
2 Domain-Driven Design (DDD) 11
2.1 Why DDD? 11
2.2 Ubiquitous Language and Domain knowledge 12
2.3 Bounded Context 12
2.4 Aggregates 13
2.5 Domain and Integration Events 13
2.6 Value objects 14
3 Description of Implementation 15
3.1 Infrastructure.Core 15
3.1.1 EntityId and IId 15
3.1.2 Entity and IEntity 16
3.1.3 Aggregate and IAggregate 17
3.1.4 EventWrapper 19
3.1.5 IEventHandler 20
3.1.6 IQuery 20
3.1.7 IRepository 21
3.1.8 Value object 21
3.2 Infrastructure.MongoDB 22
3.2.1 Repository 22
3.2.2 Query 24
3.2.3 EventHandlerBackgroundService 25
3.2.4 MongoDbContext and Context Settings 27
3.2.5 ServiceCollectionExtensions 28
4 Sample project – Library 30
4.1.1 Library.Domain 31
4.1.2 Library.ApplicationLayer 40
4.1.3 Library.Client 43
5 Conclusions 46
6 References 47

Introduction

The motivation behind this thesis is to speed up the early stages of development for .NET developers who would like to use a NoSQL database MongoDB and Domain-Driven Design (DDD) approach in their new project.

Existing frameworks like Marten[1] are more focused on persistence; other frameworks like Jasper[2] are only messages and events handling based. None of these frameworks focus on DDD and let the developer write their own little frameworks with classes and objects for the common DDD approach.

The main goal of this bachelor thesis is to create a framework **(„Framework“)** on the .NET Core platform, which will provide tools and interfaces useful for DDD projects like entity and domain aggregate objects structure. The framework will also provide a repository for basic CRUD operations, which will use a document database MongoDB 4.2.

The framework will use all main features of document database MongoDB 4.2 like advanced JSON serialization, multi-document transactions, flexible database schema, and strong consistency (MongoDB, 2019).

The framework also helps to aviod the issue of database locking known from the relational databases.

> *“Locking is a mechanism used by the SQL Server Database Engine to synchronize access by multiple users to the same piece of data at the same time.* (Microsoft, 2019)*”*

During complex objects updates with relations across the whole database, all related tables are locked until the update is finished. It is acceptable for a small amount of data, but with thousands of updates at the same time, it can be an issue. In worst cases, it can lead to very slow database responses or even deadlocks.

The framework will solve the locking issue with optimistic concurrency under the hood and a mechanism for domain event handling, which is described in detail in chapter 3.2.3.

With a document database and domain event handling combined, the framework will achieve strong consistency on the modified document and eventual consistency on related documents. It will be a perfect match for BASE[3] services.

---

[1] www.martendb.io - Polyglot Persistence for .NET Systems using the Rock Solid PostgreSQL Database
[2] www.jasperfx.github.io - Command Execution Engine and Messaging Solution for .Net Systems
[3] **B**asically **A**vailable, **S**oft state, **E**ventual consistency (Wikipedia, 2019)

To understand the conclusion and implementation details described in this bachelor thesis, the reader should have some experience with .NET platform and understand principles like dependency injection and layered architecture.

The framework will contain two main parts. The first part is **Infrastructure.Core** project which contains interfaces and some base classes. The second part is **Infrastructure**.**MongoDB** project which is dependent on both **Infrastructure.Core** and the official MongoDB .NET driver, and provides main functionality.

Part of this bachelor thesis will also include sample web application for staff in public library. This project will use layered architecture and will be divided into three parts. The first part will be library domain, second will be library application layer, and third will be a client web application, which will be using Razor pages. This sample application will use the Framework and demonstrate the basic principles of DDD.

Framework v1.0 pre-release and a sample application are available on GitHub - https://github.com/Duzij/infrastructure/releases/tag/1.0.0 .

1 Development Process Plan

In the early stages of development, it is easier to focus on Framework parts which do not have any dependencies except .NET Core - **Infrastructure.Core**. Implementation details are described in chapter 3.1.

From there, it will be useful to create a **Library.Domain** project and start describing domain objects based on classes and interfaces from **Infrastructure.Core**. To accomplish this objective, some basic business processes should be described, and the domain context should be made clear. The final implementation and business processes for sample application can be found in chapter 4.1.3.

After the domain is created, development can be moved forward to **Infrastructure.MongoDB** implementation and persistence layer and domain events handling logic can be created.

The next step is to connect the persistence layer and library domain with **Library.ApplicationLayer**. This project will also include all event handlers – one handler per event. Implementation details are described in chapter 4.1.2.

The last step will be creating a client application to test the functionality – **Library.Client**. Using a Razor Pages framework, a simple web application with a few pages will be created. More information about this application can be found in chapter 4.1.3.

2 Domain-Driven Design (DDD)

DDD is a development approach that is becoming more and more popular. This approach describes not only some best practices and tips during development, but it also gives some basic understanding of domain modeling and steps which should happen before the development process.

DDD does not describe any concrete implementations, but only approaches and techniques. Some implementation details are described in-depth, for example, in Hands-On Domain-Driven Design with .NET Core by Alexey Zimarev. This book takes a sample application and describes step-by-step process, starting from talking to customer and model creation to implementations for RavenDB and other databases. Some basic principles described in this title are also used in **Infrastructure**.**MongoDB** project.

This bachelor's thesis will touch the DDD topic only briefly, but to gain more knowledge about this development approach, two main books about DDD are highly recommended in developer communities. These are *Domain-Driven Design: Tackling Complexity in the Heart of Software* by Erick Evans (also known as Blue Book) and *Implementing Domain-Driven Design* by Vaughn Vernon (also known as Red Book).

This chapter will briefly explain why to use DDD and explain some basic DDD terms and techniques.

2.1 Why DDD?

DDD is primarily used to target projects with very complex business logic. With DDD, this complexity can be simplified by splitting business logic to different domain models, which interact with each other.

Another advantage of using domain models is that each domain can be developed by different developer teams, so it helps to distribute responsibility.

Another advantage of defining different domain models is that it is easier to understand and choose the core domain of the project. Such a domain is usually representing some part of the company that is generating the most income or contain the most valuable know-how. Based on this knowledge, the developer team can later decide what domain model should receive more attention. From here, the priorities on the project can be most aptly assigned.

2.2 Ubiquitous Language and Domain knowledge

Everything begins when the product owner and someone from developer team establish a problem to be solved with the delivered software. During these meetings, more and more details are revealed, and more unknown terms are introduced. These terms are used daily between customer company staff but are unknown for the developer team. All unknown terms are described by the product owner and are later used in code.

Such a base of terms is called ***Ubiquitous Language***. The person on the customer side who communicates with the development team, and who has the most information about the topic is called ***Domain Expert***. For larger projects and products, more domain experts could be involved.

> *"With a UBIQUITOUS LANGUAGE, conversations among developers, discussions among domain experts, and expressions in the code itself are all based on the same language, derived from a shared domain model." (Evans, 2003)*

To successfully construct Ubiquitous Language, it is not always enough to talk only to domain experts. Alexey Zimarev, in his book *Hands-On Domain-Driven Design with .NET Core*, gives an apropos piece of advice about domain knowledge and domain building.

> *"The general advice here is to talk to many different people from inside the domain, from the management of the whole organization, and from adjacent domains." (Zimarev, 2019)*

Usage of such language later in code makes code more readable even for "non-developers," which is especially practical during meetings when undertaking code demonstration to the product owner.

2.3 Bounded Context

Different domain models should have distinct explicit boundaries. Such boundaries keep the model very specific and within business processes.

> *"Set boundaries in terms of team organization, usage within specific parts of the application, and physical manifestations such as code bases and database schemas. Keep the model strictly consistent within these bounds, but don't be distracted or confused by issues outside." (Evans, 2003)*

To have a clear bounded context will also help to prevent projects in the future from unexpected growth and, as a consequence, the complexity of the whole project.

2.4 Aggregates

An aggregate is a high-level object in domain model composition. Such objects aggregate entity objects with common context and business logic.

"An AGGREGATE is a cluster of associated objects that we treat as a unit for the purpose of data changes. Each AGGREGATE has a root and a boundary. The boundary defines what is inside the AGGREGATE. The root is a single, specific ENTITY contained in the AGGREGATE." (Evans, 2003)

Entities inside aggregates should be isolated from the outside world. Such a rule helps to keep a consistent state of these entities and prevent deadlocks on the persistence layer.

2.5 Domain and Integration Events

It is a common requirement in DDD projects that one aggregate needs to be informed about the changes in another aggregate, and consequently react to this change. This behavior can be executed using domain events.

"An event is something that has happened in the past. A domain event is something that happened in the domain that you want other parts of the same domain (in-process) to be aware of. The notified parts usually react somehow to the events." (Microsoft, 2018)

Usually, domain events are raised inside the aggregate object and are immediately processed. Aggregate code is the only place where event collection is changing, so event collection shall be immutable. The context of handling is usually the same process and the same domain.

It is different from integration events, which are notifying different parts of application about changes across all domains.

"Semantically, domain, and integration events are the same thing: notifications about something that just happened. However, their implementation must be different. Domain events are just messages pushed to a domain event dispatcher, which could be implemented as an in-memory mediator based on an IoC container or any other method.

On the other hand, the purpose of integration events is to propagate committed transactions and updates to additional subsystems, whether they are other microservices, Bounded Contexts, or even external applications. Hence, they should occur only if the entity is successfully persisted, otherwise, it's as if the entire operation never happened." (Microsoft, 2018)

Handling logic is separated per event – every event has its own handler. It helps to keep handling logic clean and makes it easier to add new event handlers during the project lifetime.

2.6 Value objects

A value object is not a domain-driven design term, but a concept used across technologies and programming languages. However, it is included in this introduction chapter about domain-driven design terminology because it is often used in DDD projects and, as such, is part of **Infrastructure.Core**.

Value object is a structure which, when compared to other value objects, are compared not by reference, as it is by default in modern programming languages, but by its inner value. There are a couple of properties needed for a successful value object.

"There are two main characteristics for value objects: They have no identity. They are immutable." (Microsoft, 2020)

3 Description of Implementation

This chapter describes classes, interfaces, and functionality in **Infrastructure.Core** and **Infrastructure.MongoDB** projects.

3.1 Infrastructure.Core

The project itself has no dependencies and uses only .NET Core. It has few implementation classes like `Entity`, `DomainAggregate` or `ValueObject`, but overall, this project contains only interfaces, which makes it independent from the database engine and leaves a possibility to create other implementations for different database engines if needed.

3.1.1 EntityId and IId

The interface below describes a simple typed identifier object with Value property (see Code 3-1). Typed identifiers are needed for integrity purposes and are very useful to set constraints in the business layer code. It is more obvious from code in the sample application layer in chapter 4.1.1. Domain-driven design does not require typed identifiers, but in general, typed identifiers are quite popular as they help with avoiding mistakes when some identifier is used for an entity of a different type.

Code 3-1

```
1: namespace Infrastructure.Core
2: {
3:     public interface IId<TKey>
4:     {
5:         public TKey Value { get; }
6:     }
7: }
```

`IId` interface implementation is also very simple (see Code 3-2). In addition to `IId` interface, `EntityId` class is also implementing `ValueObject`. In this case, the overridden method `GetAtomicValues` returns only `Value` property, so in the event when two entity identifiers needs to be compared, only property `Value` is compared. `ValueObject` implementation is available in chapter 3.1.8.

Code 3-2

```
7: namespace Infrastructure.Core
8: {
9:     public class EntityId<T> : ValueObject, IId<string>
10:     {
11:         public string Value { get; private set; }
12:         public EntityId(string value)
13:         {
14:             Value = value;
15:         }
16:         protected override IEnumerable<object> GetAtomicValues()
17:         {
18:             return new List<object>() { this.Value };
19:         }
20:     }
21: }
```

3.1.2 Entity and IEntity

`IEntity` interface definition includes typed identifier property named `Id` with only getter (see Code 3-3Code 3-4).

Code 3-3

```
1: namespace Infrastructure.Core
2: {
3:     public interface IEntity<TKey>
4:     {
5:         public IId<TKey> Id { get; }
6:     }
7: }
```

`Entity` implementation is a simple abstract class with a typed identifier (see Code 3-4Code 3-4). It is important to notice that `Id` property has only getter declaration and that its implementations setter has `protected` modifier. It helps to set object boundaries and prevent situations when `Id` is changed from the outside.

Code 3-4

```
7: namespace Infrastructure.Core
8: {
9:     public abstract class Entity<TKey> : IEntity<TKey>
10:    {
11:         public IId<TKey> Id { get; protected set; }
12:    }
13: }
```

3.1.3 Aggregate and IAggregate

`IAggregate` is an interface which contains a definition of a functionalities for high-level DDD object (see Code 3-5). This interface inherits `IEntity` interface (see Code 3-3) which require typed identifier property `Id`.

Code 3-5

```
3: namespace Infrastructure.Core
4: {
5:     public interface IAggregate<TKey> : IEntity<TKey>
6:     {
7:         void CheckState();
8:         public string Etag { get; }
9:         void RegenerateEtag();
10:        bool Equals(object other);
11:        int GetHashCode();
12:        public IEnumerable<object> GetEvents();
13:     }
14: }
```

Members `Equals` and `GetHashCode` are necessary for aggregate comparison.

Domain events constrains described in chapter 2.5 have been explicitly discussed. Domain Events shall be added only inside the aggregate object and returned events shall be immutable. In `IAggregate` interface `GetEvents` method and is used to access stored aggregate events from outside. To prevent someone from adding new events externally, `GetEvents` method uses non-void return type `IEnumerable<object>`[4] (see Code 3-6).

`Aggregate` is as an abstract class with common logic for all high-level object in DDD (see Code 3-6). Because domain events are a key functionality for the Framework, a private list of objects is instantiated when the aggregate object is created. New domain events can be added to list with `AddEvent` method.

`Etag` property is used for optimistic concurrency implementation in repository. More information about that can be found in chapter 3.2.1.

In complex business logic more than one valid state of object could be possible. Abstract member `CheckState` define these valid states. This method is called automatically before any

---

[4] IEnumerable available methods - https://docs.microsoft.com/en-us/dotnet/api/system.collections.ienumerable?view=netframework-4.8#extension-methods

CRUD operation in repository, to check whether inner object state is valid. Because the method is abstract, it is necessary for the inherited object to implement this method.

Code 3-6

```
5: namespace Infrastructure.Core
6: {
7:     public abstract class Aggregate<TKey> : Entity<TKey>, IAggregate<TKey>
8:     {
9:         public Aggregate()
10:         {
11:             Events = new List<object>();
12:         }
13:
14:         public string Etag { get; protected set; }
15:
16:         public void RegenerateEtag()
17:         {
18:             Etag = Guid.NewGuid().ToString();
19:         }
20:
21:         private List<object> Events { get; set; }
22:
23:         public abstract void CheckState();
24:
25:         public override bool Equals(object other)
26:         {
27:             if (ReferenceEquals(null, other)) return false;
28:             if (ReferenceEquals(this, other)) return true;
29:             return this == other;
30:         }
31:
32:         public override int GetHashCode()
33:         {
34:             return base.GetHashCode();
35:         }
36:
37:         protected void AddEvent(object @event)
38:         {
39:             CheckState();
40:             if (Events == null)
41:             {
42:                 Events = new List<object>();
43:             }
44:             Events.Add(@event);
45:         }
```

```
46:
47:         public IEnumerable<object> GetEvents()
48:         {
49:             if (Events == null)
50:             {
51:                 return new List<object>();
52:             }
53:             return Events;
54:         }
55:     }
56: }
```

In the sample web application aggregates like Autor, Book, Library record, or User are given (see chapter 4.1.1).

3.1.4 EventWrapper

`EventWrapper` is used to save event, its type and other useful metadata about raised event to `__event` collection in database (see Code 3-7). For each aggregate event, an `EventWrapper` object is created. It helps not only store useful information about created event, but also keep same database model for different events. `EventWrapper` stores a reference to the aggregate object where it was raised, time when it was created, event type and event object itself in binary format. Every created event is places inside its own `EventWrapper` object during create or update operations in repository. More information about event storing can be found in chapter 3.2.1.

Code 3-7

```
5: namespace Infrastructure.Core
6: {
7:     public class EventWrapper
8:     {
9:         public string Id { get; private set; }
10:
11:         public string EntityId { get; private set; }
12:
13:         public string EventType { get; private set; }
14:
15:         public object EventValue { get; private set; }
16:
17:         public DateTime CreatedTime { get; private set; }
18:
19:         public EventWrapper(string eventId, Type eventType, object @event, string
entityId)
```

```
20:         {
21:             Id = eventId;
22:             EventType = eventType.AssemblyQualifiedName;
23:             EventValue = @event;
24:             EntityId = entityId;
25:             CreatedTime = DateTime.UtcNow;
26:         }
27:
28:     }
29: }
```

3.1.5 IEventHandler

IEventHandler interface is a crucial interface for event handling in Framework. An interface contains only one method `Handle`, wherein interface implementation shall describe handling logic. It is necessary to know the exact type of event a handler is working with, so the interface contains a generic parameter `TEvent`. Method Handle has a parameter with the same type (see Code 3-8).

Code 3-8

```
6: namespace Infrastructure.Core
7: {
8:     public interface IEventHandler<TEvent>
9:     {
10:         Task Handle(TEvent @event);
11:     }
12: }
```

3.1.6 IQuery

`IQuery` is a simple interface for query objects, which asynchronously reads data from the database (see Code 3-9). `Query` object implementation is dependent on a database engine so that it can be found in **Infrastructure.MongoDB** project and described in chapter 3.2.2.

Code 3-9

```
6: namespace Infrastructure.Core
7: {
8:     public interface IQuery<TResult>
9:     {
10:         public abstract Task<IList<TResult>> GetResultsAsync();
11:     }
12: }
13:
```

3.1.7 IRepository

`IRepository` is an interface that defines interaction with the persistence layer (see Code 3-10). To make both this interface and its implementation independent from the identifier type or entity type, it has two generic parameters. The only condition is that the entity should have a typed identifier, so it must inherit `IEntity` interface. Same identifier is used in methods `GetByIdAsync` and `RemoveAsync`.

Code 3-10

```
5: namespace Infrastructure.Core
6: {
7:     public interface IRepository<T, TKey> where T : IEntity<TKey>
8:     {
9:          ...
14:         Task InsertNewAsync(T entity);
15:         ...
22:         Task ReplaceAsync(T entity);
23:         ...
30:         Task ModifyAsync(Action<T> modifyLogic, IId<TKey> id);
31:         ...
37:         Task<T> GetByIdAsync(IId<TKey> id);
38:         ...
44:         Task RemoveAsync(IId<TKey> id);
45:     }
46: }
```

Besides basic repository methods like `InsertNewAsync`, `RemoveAsync`, and `GetByIdAsync`, two methods for modification are introduced.

`ReplaceAsync` takes the whole entity object as a parameter, finds it, and replaces it in the database. Such an update operation is not very common in real-world applications but can be useful for some types of aggregates where versioning and concurrency control does not place like for example user profile picture.

`ModifyAsync` also updates the data, but does so using optimistic concurrency control. For more about inner logic and implementation, see chapter 3.2.1, where this process is described step by step.

3.1.8 Value object

`ValueObject` is a very simple value object implementation referenced from Microsoft documentation (see Code 3-11). This implementation of value object is an abstract class with abstract method `GetAtomicValues`. When inherited, this method describes what properties

should be compared. For example, for typed identifier `EntityId` it is `Value` property (see Code 3-2)

Code 3-11

```
10:     /// <summary>
11:     /// Referenced from https://github.com/dotnet-
architecture/eShopOnContainers/blob/dev/src/Services/Ordering/Ordering.Domain/SeedWork/Val
ueObject.cs
12:     /// </summary>
13:     public abstract class ValueObject
14:     {
15:         public static bool operator ==(ValueObject left, ValueObject right)
16:         {
17:             if (ReferenceEquals(left, null) ^ ReferenceEquals(right, null))
18:             {
19:                 return false;
20:             }
21:             return ReferenceEquals(left, null) || left.Equals(right);
22:         }
23:
24:         public static bool operator !=(ValueObject left, ValueObject right)
25:         {
26:             return !(left == right);
27:         }
28:
29:         protected abstract IEnumerable<object> GetAtomicValues();
```

3.2 Infrastructure.MongoDB

3.2.1 Repository

The repository takes care of storing, updating, and deleting objects from the MongoDB database. To meet DDD constraints and keep aggregates boundaries, whole domain aggregate objects are stored and retrieved from the database. It is a great fit for document databases such as MongoDB, wherein the database model is denormalized and complex objects with many child elements can be stored.

“Aggregates are the basic element of transfer of data storage - you request to load or save whole aggregates. Transactions should not cross aggregate boundaries” (Fowler, 2020).

To store aggregates, transaction logic must be applied. The aggregate and its events must each be individually saved. The reason behind this, is that separate document for events is used as storage for all events from all aggregates in application. After all application event are stored separately, the background service responsible for events handling can then

subscribe to changes on that event document and handle all events one by one. This process is described in detail in chapter 3.2.3. The event document is called `__events` by default.

To apply transaction logic, Repository use MongoDB 4.0 feature – **multi-document transactions** (see Code 3-12).

Code 3-12

```
41:      using (var session = dbContext.Database.Client.StartSession())
42:      {
43:                 try
44:                 {
45:                     await session.WithTransaction(
46:                     async (s, ct) =>
47:                     {
48                          //store aggregate and domain events
49:                     });
```

This transaction logic, which is also required to store domain events, is used in `InsertNewAsync`, `ReplaceAsync` and `ModifyAsync` methods.

As mentioned, the `ModifyAsync` method uses **optimistic concurrency** to store domain aggregates. It uses `Etag` property as a so-called **concurrency token**. This property is generated when aggregate is created and regenerated every time the aggregate is modified. Sometimes this property is called **"timestamp"** or **"row version"**, but logic is always identical. (Microsoft, 2018)

To successfully update an aggregate, `Etag` property, which was read from the database before the modification and `Etag`, which is read from the database after modifications are applied, should match. Such an approach is referred to as optimistic, as it optimistically assumes the concurrency token will almost always match and update operation will be successful.

However, if these two properties do not match, then it means that during modification, a separate process must have performed other changes to the same entity and thereby regenerated concurrency token.

If two processes start an update, then the modification which finishes first will automatically be saved, while the second process is terminated and receives an error message or in case of `Repository`, second process is forced to retry the modification attempt. The fully finished modifications will regenerate the concurrency token.

The `Repository` performs a few steps to implement optimistic concurrency (see Code 3-13). Before the aggregate is modified, it is retrieved from the database by its identifier. Next, the old aggregate `Etag` is saved, and a new one is generated. Then the retrieved object is modified. The next step is to find matching aggregate not only by its ID but also saved `Etag`. The last step is replacing the modified aggregate. If the replace method modifies less than one aggregate, then the whole process is repeated.

Code 3-13

```
109:     do
110:     {
111:        var filter = Builders<T>.Filter.Eq(MongoDefaultSettings.IdName, id.Value);
112:        foundAggregate = await GetFirstFromCollectionAsync(entityCollection, filter);
113:        var version = foundAggregate.Etag;
114:        foundAggregate.Etag = Guid.NewGuid().ToString();
115:
116:        modifyLogic(foundAggregate);
117:        filter = filter & Builders<T>.Filter.Eq(MongoDefaultSettings.EtagName,
version);
118:
119:        result = await entityCollection.ReplaceOneAsync(filter, foundAggregate, new
ReplaceOptions { IsUpsert = false });
120:
121:     } while (result.ModifiedCount != 1);
```

MongoDB, like any other database, has a pool of workers that query a database. According to documentation the default pool size is five workers. This property can be changed with connection string property `poolSize`. If all pool workers are waiting or executing a job, `MongoWaitQueueFullException` is thrown. During performance testing of the first version of the repository, scenarios when such exception was thrown, where quite often if the count of processes that try to modify an aggregate simultaneously, was higher than 500.

To prevent this scenario, the `Polly` package was added to **Infrastructure.MongoDB**. This package easily facilitates adding exception handling policy that automatically retries code execution after a delay period (see Code 3-14). In case exception `MongoWaitQueueFullException` is thrown, retry attempt number is saved, and a number of milliseconds are calculated. Number of milliseconds is equal to attempt number raised to the power of two. The method is then put on hold and is executed only after a number of milliseconds have elapsed.

Code 3-14

```
105:  await Policy
106:             .Handle<MongoWaitQueueFullException>()
107:             .WaitAndRetryForeverAsync(retryAttempt =>
TimeSpan.FromMilliseconds(Math.Pow(2, retryAttempt)))
108:             .ExecuteAsync(async () =>
109:             {
```

3.2.2 Query

`Query` object is a simple abstract class with only one abstract method `GetResultsAsync` (see Code 3-15). All query read something from database, so in constructor reference to database

context is injected. More about database context and `IMongoDbContext` interface can be found in chapter 3.2.4. Further, some query object examples can be found in chapter 4.1.2 (see Code 4-8).

Code 3-15

```
8: namespace Infrastructure.MongoDb
9: {
10:     public abstract class Query<TResult> : IQuery<TResult>
11:     {
12:         public readonly IMongoDbContext dbContext;
13:
14:         public Query(IMongoDbContext dbContext)
15:         {
16:             this.dbContext = dbContext;
17:         }
18:
19:         public abstract Task<IList<TResult>> GetResultsAsync();
20:     }
21: }
```

3.2.3 EventHandlerBackgroundService

`EventHandlerBackgroundService` is the portion of Framework responsible for event handling. This hosted service which is subscribed on `__events` document changes. Service starts with web application and is never stopped. It automatically tries to find corresponding event handler based on saved event type and execute `Handle` method. In case event handler is not found and the exception is thrown, service logs this exception and wait for next changes in `__events` document. It guarantees, that background service will be always running.

It uses abstract base class `BackgroundService` provided by Microsoft in `Microsoft.Extensions.Hosting` assembly. To automatically start with web application, it shall be also registered (see Code 3-20). Method `ExecuteAsync` is automatically executed after application starts (see Code 3-16).

Code 3-16

```
28: protected override async Task ExecuteAsync(CancellationToken stoppingToken)
29: {
30:     var collection =
dbContext.Database.GetCollection<EventWrapper>(MongoDefaultSettings.EventsDocumentName);
31:     var pipeline = new EmptyPipelineDefinition<ChangeStreamDocument<EventWrapper>>()
32:     .Match(x => x.OperationType == ChangeStreamOperationType.Insert);
33:     using (var cursor = await collection.WatchAsync(pipeline))
34:     {
35:         await cursor.ForEachAsync(change =>
```

```
36:         {
37:             logger.LogInformation($"{change.CollectionNamespace.FullName} changed.
Processing change.");
38:             try
39:             {
40:                 ProcessEvent(change);
41:             }
42:             catch (Exception ex)
43:             {
44:                 logger.LogError(ex.Message, ex);
45:             }
46:         });
47:     }
48: }
```

`EventHandlerBackgroundService` is using MongoDB change streams to track every insert operation on `__event` collection. To use such feature at least one MongoDB replica set should exist. Sample configuration file and prerequisites are described in detail in chapter 4.1.3.

When a new event is inserted into the document and `ProcessEvent` method is executed, then a few steps are performed (see Code 3-17). The service automatically creates a generic handler type, based on the event type saved in `EventType` property. Next, this handler is retrieved from .NET Core service provider and method `Handle` is executed through .NET reflection (Microsoft, 2017).

Event handlers used in sample application can be found in chapter 4.1.2.

Code 3-17

```
50:     private void ProcessEvent(ChangeStreamDocument<EventWrapper> change)
51:     {
52:     var @event = change.FullDocument;
53:
54:     using (var factory = serviceScopeFactory.CreateScope())
55:     {
56:         Type handlerGenericType =
typeof(IEventHandler<>).MakeGenericType(Type.GetType(@event.EventType));
57:         var service = factory.ServiceProvider.GetRequiredService(handlerGenericType);
58:         HandleEvent(service, @event);
59:     }
60:
61: }
62:
63: private void HandleEvent(object service, EventWrapper @event)
64: {
65:     var methodInfo = service.GetType().GetMethod("Handle");
```

```
66:     var eventParameter = new object[1] { @event.EventValue };
67:     methodInfo.Invoke(service, eventParameter);
68: }
```

3.2.4 MongoDbContext and Context Settings

MongoDB has an official .NET driver which is also used in **Infrastructure.MongoDB** project. Working with the plain .NET driver is an option, but to keep the code clean and reduce code duplication, usually some abstraction is needed. Such abstraction is `MongoDbContext` (see Code 3-18) and its settings in `MongoDbSettings` (see Code 3-19).

Code 3-18

```
6: namespace Infrastructure.MongoDB
7: {
8:     public class MongoDbContext : IMongoDbContext
9:     {
10:         public MongoDbContext(IMongoDbSettings settings)
11:         {
12:             var client = new MongoClient(settings.ConnectionString);
13:             Database = client.GetDatabase(settings.DatabaseName);
14:         }
15:
16:         public IMongoDatabase Database { get; }
17:
18:         public IMongoCollection<T> GetCollection<T>() =>
Database.GetCollection<T>(MongoUtils.GetCollectionName<T>());
19:
20:     }
21: }
```

`MongoDbContext` class loads saved connection string and database name from settings and initiate `Database` property.

Method `GetCollection` is used to return saved MongoDB documents as collections. This method is often used in queries (see Code 4-8).

Code 3-19

```
2: namespace Infrastructure.MongoDB
3: {
4:     public class MongoDbSettings : IMongoDbSettings
5:     {
6:
```

```
7:         public MongoDbSettings(string connectionString, string databaseName)
8:         {
9:             ConnectionString = connectionString;
10:             DatabaseName = databaseName;
11:         }
12:
13:         public string ConnectionString { get; set; }
14:
15:         public string DatabaseName { get; set; }
16:     }
17: }
```

`MongoDbSettings` is a class for storing MongoDB connection string and database name. More information about connection string formats can be found on official documentation page www.docs.mongodb.com/manual/reference/connection-string.

3.2.5 ServiceCollectionExtensions

`ServiceCollectionExtensions` is a static class for registering all **Infrastructure.MongoDB** objects and services in *.NET IoC container*[5] (see Code 3-20).

`EventHandlerBackgroundService` is registered here, as well as in the generic `IRepository` interface. Thanks to *Scrutor*[6] framework all new event handlers and query objects are also automatically registered and ready to use.

This class makes it easier for the developer to use the Framework. All needed functionality can be added with one line of code in `Startup.cs` class. An example is described in chapter 4.1.2.

Code 3-20

```
27:         public static IServiceCollection AddMongoDbInfrastructure(this
IServiceCollection services, MongoDbSettings settings = null)
28:         {
29:             services.AddSingleton<IMongoDbSettings>(settings);
30:             AddMongoDbInfrastructureServices(services);
31:             return services;
32:         }
33:
```

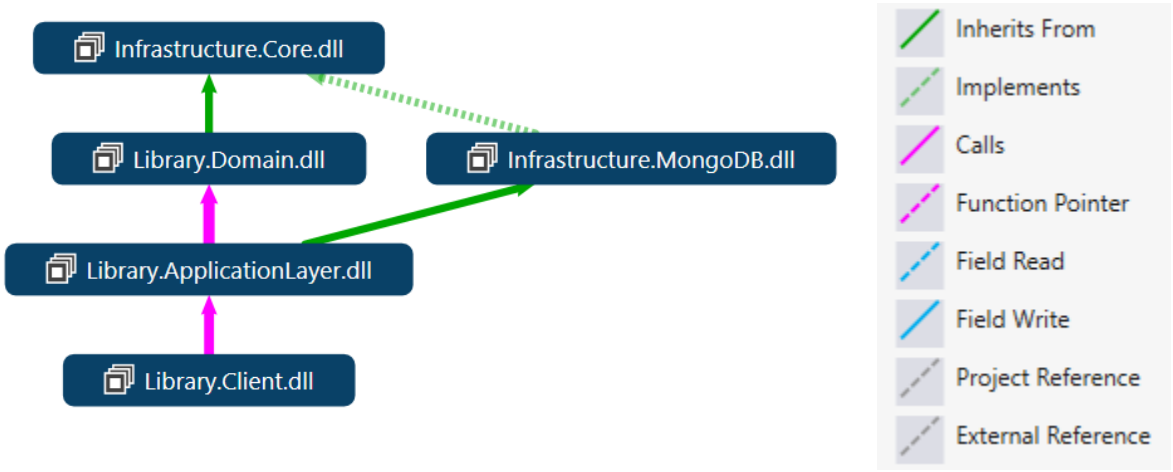
[5] ASP.NET Core supports the dependency injection (DI) software design pattern, which is a technique for achieving Inversion of Control (IoC) between classes and their dependencies. (Microsoft, 2020)
[6] Scrutor - Assembly scanning and decoration extensions for Microsoft.Extensions.DependencyInjection (https://github.com/khellang/Scrutor)

```
34:         private static void AddMongoDbInfrastructureServices(IServiceCollection
services)
35:         {
36:             services.AddTransient<IMongoDbContext, MongoDbContext>();
37:             services.AddHostedService<EventHandlerBackgroundService>();
38:
39:             services.AddTransient(typeof(IRepository<,>), typeof(Repository<,>));
40:
41:             services.Scan(scan => scan.FromApplicationDependencies()
42:             .AddClasses(classes => classes.AssignableTo(typeof(IEventHandler<>)))
43:             .AsImplementedInterfaces()
44:             .WithTransientLifetime());
45:
46:             services.Scan(scan => scan.FromApplicationDependencies()
47:             .AddClasses(classes => classes.AssignableTo(typeof(Query<>)))
48:             .AsSelf()
49:             .WithTransientLifetime());
50:         }
```

4 Sample project – Library

This chapter describes three parts of a sample project that was added to this bachelor thesis as a Framework proof of concept. Diagram 1 shows all project dependencies.

Diagram 1



All three projects are part of a simple web application for library staff to cover the following requirements:

- Library Users
  - Staff can register and later edit library users with name, surname, and email.
  - Staff can set registered users as banned. If the user set at banned, no library records with this user shall be created.
  - When the user name or surname is updated, changes shall be populated to relevant library records.
- Authors
  - Staff can create and edit records about book authors with author name and surname.
  - When the author name or surname is updated, relevant books shall be updated.
- Books
  - Staff can see all books in database and track its number on the shelf (amount) on one page.
  - Staff can create and edit records about books, which will include book name, description, and existing author.
  - Staff can add the number of books to existing book records.
  - When the book name or author is changed, relevant library records and authors shall be updated.
- Library records

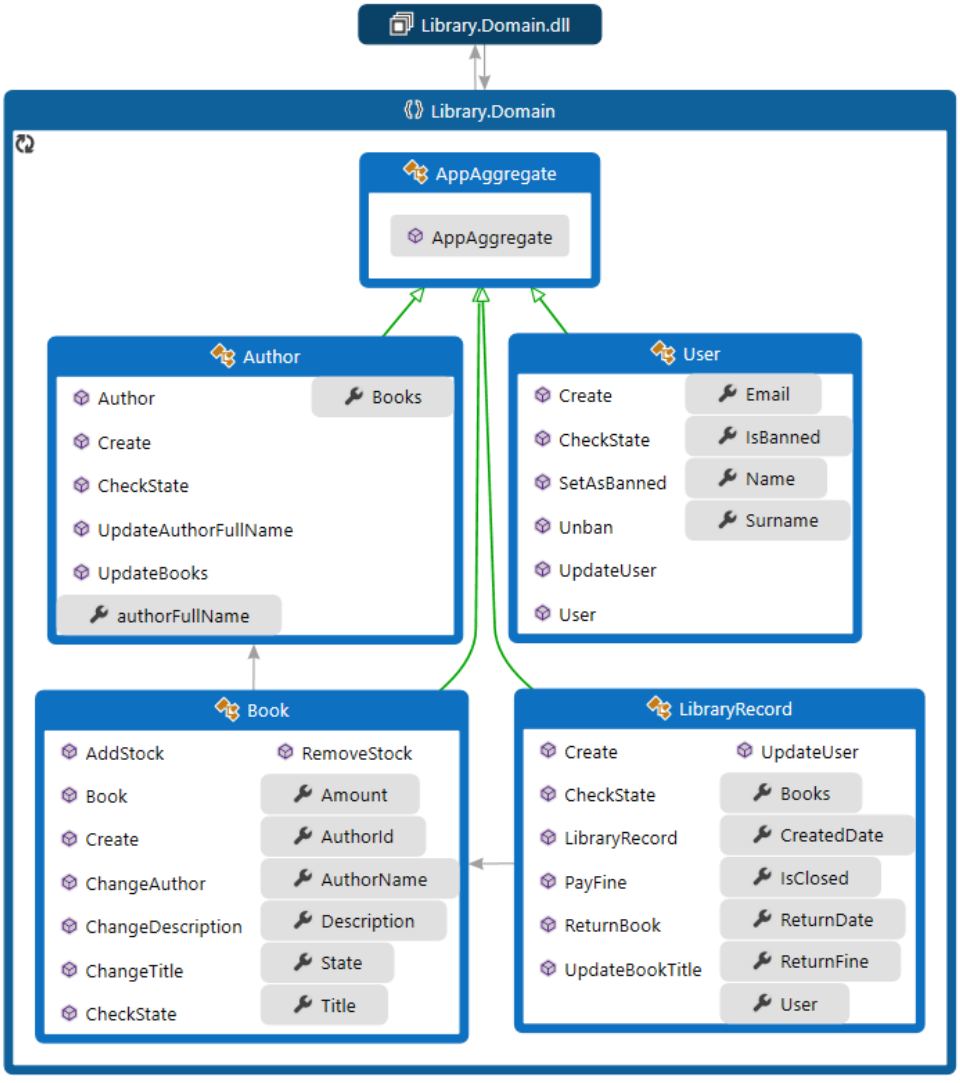
- Staff can create library records for existing users and books, which has a number larger than 0.
- Users are allotted one week to return all books. If this period expires, then a fine will be automatically charged.
- Users may return books individually, or all at once.
- The library record is set as closed when all books are returned.
- When a library record is created, its book amount is reduced.
- When a library record is closed, its book amount is increased.
- Staff must be able to see a history of closed library records on a separate page.

4.1.1 Library.Domain

Library.Domain is a project where all library domain aggregates, events, and typed identifiers are described. As seen in Diagram 1, the only dependency of this project is **Infrastructure.Core**.

This chapter will focus on the most complex part of the domain, which is aggregates. Diagram 2 shows properties and methods of all domain aggregates for library domain.

Diagram 2



Library.Domain is using identifiers with type string. `AppAggregate` object is a base class for all aggregates in the domain and set identifier value type - string (see Code 4-1).

Code 4-1

```
6: namespace Library.Domain
7: {
8:     public abstract class AppAggregate : Aggregate<string>
9:     {
10:    }
11: }
```

Another common feature is the factory `Create` method for all aggregates. Such method makes is easier to restrict unique typed identifier initialization. The factory method is similar for all aggregates. An example of such method for library record is as follows (see Code 4-2).

Code 4-2

```
19: public static LibraryRecord Create(UserRecord user, List<BookRecord> books)
20: {
21:       var record = new LibraryRecord(TypedId.GetNewId<LibraryRecordId>(), user, books);
22:       return record;
23: }
```

`TypeId.GetNewId` method return typed identifier with the unique `Value` property. Value property is generated from *Guid*[7] and cannot be replaced.

Another set of common features for all aggregate objects are `private` setter modifiers for all properties. Using these makes it impossible to change these properties from outside the aggregate object. The significance behind this is explained in detail in chapter 2.4.

User Aggregate

To meet all library requirements, `User` aggregate is following (see Code 4-3). This aggregate will raise only one event – `UserUpdated` when the user is updated. As a result, the event handler for this event will automatically update the username and surname in all user library records.

---

[7] **GUID** (or UUID) is an acronym for 'Globally Unique Identifier' (or 'Universally Unique Identifier') (Wikipedia, 2020)

Code 4-3

```
5: namespace Library.Domain
6: {
7:     public class User : AppAggregate
8:     {
9:         public bool IsBanned { get; private set; }
10:         public string Name { get; private set; }
11:         public string Surname { get; private set; }
12:         public string Email { get; private set; }
13:
14:         public static User Create(string name, string surname, string email)
15:         {
16:             return new User(TypedId.GetNewId<UserId>(), name, surname, email);
17:         }
18:
19:         private User(UserId id, string name, string surname, string email)
20:         {
21:             Id = id;
22:             IsBanned = false;
23:             Name = name;
24:             Surname = surname;
25:             Email = email;
26:         }
27:
28:         public void SetAsBanned()
29:         {
30:             this.IsBanned = true;
31:         }
32:
33:         public void UpdateUser(string name, string surname, string email)
34:         {
35:             var oldUser = new UserRecord(this.IsBanned, this.Name, this.Surname,
this.Email);
36:             this.Name = name;
37:             this.Surname = surname;
38:             this.Email = email;
39:             var updatedUser = new UserRecord(this.IsBanned, name, surname, email);
40:             AddEvent(new UserUpdated(oldUser, updatedUser));
41:         }
42:
43:         public override void CheckState()
44:         {
45:             if (string.IsNullOrWhiteSpace(Name) ||
46:                 string.IsNullOrWhiteSpace(Surname) ||
47:                 string.IsNullOrWhiteSpace(Email))
```

```
48:             {
49:                 throw new InvalidEntityStateException();
50:             }
51:         }
52:
53:         public void Unban()
54:         {
55:             this.IsBanned = false;
56:         }
57:     }
58: }
```

Author Aggregate

`Author` aggregate object allow to set books and update name and surname properties. This object raises an `AuthorUpdated` event if author is updated. Event handler for this event update all book records for this author.

```
5: namespace Library.Domain
6: {
7:     public class Author : AppAggregate
8:     {
9:         public string Name { get; private set; }
10:         public string Surname { get; private set; }
11:
12:         public IList<AuthorBookRecord> Books { get; private set; }
13:
14:         public static Author Create(string name, string surname)
15:         {
16:             var author = new Author(TypedId.GetNewId<AuthorId>(), name, surname);
17:             return author;
18:         }
19:
20:         private Author(AuthorId id, string name, string surname)
21:         {
22:             Id = id;
23:             Name = name;
24:             Surname = surname;
25:             this.Books = new List<AuthorBookRecord>();
26:         }
27:
28:
29:         public override void CheckState()
30:         {
```

```
31:             if (Id == null || string.IsNullOrWhiteSpace(Id.Value) ||
string.IsNullOrWhiteSpace(Name) || string.IsNullOrWhiteSpace(Surname) || Books == null)
32:             {
33:                 throw new InvalidEntityStateException();
34:             }
35:         }
36:
37:         public void UpdateBooks(IList<AuthorBookRecord> books)
38:         {
39:             this.Books = books;
40:             CheckState();
41:         }
42:
43:         public void UpdateAuthor(string name, string surname)
44:         {
45:             var oldName = new AuthorFullName(this.Name, this.Surname);
46:             var newName = new AuthorFullName(name, surname);
47:             if (oldName != newName)
48:             {
49:                 this.Name = name;
50:                 this.Surname = surname;
51:                 AddEvent(new AuthorUpdated(oldName, newName, (AuthorId)Id));
52:             }
53:         }
54:     }
55: }
```

Book Aggregate

Book aggregates need a title, description, and existing author to be created. To meet all of these requirements, the book aggregate correspond with methods and properties that allow to add or remove stock and update title, description, or author. Book aggregate also raise `BookCreated` event when created and notify corresponding author, it also raises `BookAuthorNameChanged` event when Author is updated and `BookTitleChanged` when title is changed (see Code 4-4).

Code 4-4

```
6: namespace Library.Domain
7: {
8:     public class Book : AppAggregate
9:     {
10:         public BookAmount Amount { get; private set; }
11:         public BookTitle Title { get; private set; }
12:         public string Description { get; private set; }
```

```
13:         public AuthorId AuthorId { get; private set; }
14:         public AuthorFullName AuthorName { get; private set; }
15:         public BookState State { get; private set; }
16:
17:         public static Book Create(string title, string description, AuthorId authorId,
AuthorFullName authorName)
18:         {
19:             var book = new Book(
20:                 TypedId.GetNewId<BookId>(), title, description, authorId, authorName);
21:             book.AddEvent(new BookCreated((BookId)book.Id, title, description,
authorId));
22:             return book;
23:         }
24:
25:         private Book(BookId id, string title, string description, AuthorId authorId,
AuthorFullName authorName)
26:         {
27:             Id = id;
28:             Title = new BookTitle(title);
29:             Description = description;
30:             AuthorId = authorId;
31:             AuthorName = authorName;
32:             State = BookState.InDatabase;
33:             Amount = new BookAmount(0);
34:         }
35:
36:         public void ChangeAuthor(AuthorFullName author, AuthorId authorId)
37:         {
38:             if (this.AuthorName == author)
39:             {
40:                 throw new InvalidOperationException("Author name was already
changed");
41:             }
42:             AddEvent(new BookAuthorNameChanged(Id.Value, this.Title.Value,
authorId.Value, AuthorId.Value));
43:             this.AuthorId = authorId;
44:             this.AuthorName = author;
45:         }
46:
47:         public void AddStock(BookAmount amount)
48:         {
49:             this.Amount = new BookAmount(this.Amount.Amount + amount.Amount);
50:             this.State = BookState.InStock;
51:         }
52:
```

```
53:         public void RemoveStock(BookAmount amount)
54:         {
55:             this.Amount = new BookAmount(this.Amount.Amount - amount.Amount);
56:             if (this.Amount.Amount == 0)
57:             {
58:                 this.State = BookState.InDatabase;
59:             }
60:         }
61:
62:         public override void CheckState()
63:         {
64:             if (Id == null || Amount == null ||
65:                 AuthorId == null ||
66:                 Title == null ||
67:                 AuthorName == null)
68:             {
69:                 throw new InvalidEntityStateException();
70:             }
71:         }
72:
73:         public void ChangeTitle(BookTitle bookTitle)
74:         {
75:             AddEvent(new BookTitleChanged(Id.Value, this.Title.Value,
bookTitle.Value));
76:             this.Title = bookTitle;
77:         }
78:
79:         public void ChangeDescription(string description)
80:         {
81:             this.Description = description;
82:         }
83:     }
84: }
```

Library Record Aggregate

According to library requirements, to successfully create a library record, all books should have enough amount of stock and user who is not set as banned. It also raises `BookAddedToLibraryRecord` event for each book. The handler for this event automatically updates the book stock and reduces it to number of books which were lent. For each book which is returned, it raises `BookRemovedFromLibraryRecord` event. The handler for this event increases the book stock to number of books which were returned (see Code 4-5). Applying such logic, users could return books individually, or if they borrow multiple books with same

title, they would be able to return them one by one. If there is any fine, it is possible to pay it only once all books are returned.

Code 4-5

```
6: namespace Library.Domain
7: {
8:     public class LibraryRecord : AppAggregate
9:     {
10:         public bool IsClosed { get; private set; }
11:         public UserRecord User { get; private set; }
12:         public List<BookRecord> Books { get; private set; }
13:
14:         public DateTime CreatedDate { get; private set; }
15:         public DateTime ReturnDate => CreatedDate.AddDays(7);
16:
17:         public ReturnFine ReturnFine { get; private set; }
18:
19:         public static LibraryRecord Create(UserRecord user, List<BookRecord> books)
20:         {
21:             var record = new LibraryRecord(TypedId.GetNewId<LibraryRecordId>(), user,
books);
22:             return record;
23:         }
24:
25:         private LibraryRecord(LibraryRecordId id, UserRecord user, List<BookRecord>
books)
26:         {
27:             Id = id;
28:             if (user.IsNotBanned)
29:             {
30:                 User = user;
31:             }
32:             else
33:             {
34:                 throw new InvalidEntityStateException();
35:             }
36:             this.CreatedDate = DateTime.UtcNow;
37:             this.ReturnFine = new ReturnFine(0);
38:             Books = books;
39:             foreach (var book in books)
40:             {
41:                 AddEvent(new BookAddedToLibraryRecord(book.BookId, id,
book.BookAmount.Amount));
42:             }
43:         }
```

```
44:
45:         public void UpdateUser(UserRecord userRecord)
46:         {
47:             if (userRecord.IsNotBanned)
48:             {
49:                 this.User = userRecord;
50:             }
51:         }
52:
53:         public void ReturnBook(BookId bookId, BookAmount amount)
54:         {
55:             var book = Books.First(a => a.BookId == bookId);
56:             if (book.BookAmount == amount)
57:             {
58:                 Books.RemoveAll(a => a.BookId == bookId);
59:             }
60:             else if (book.BookAmount > amount)
61:             {
62:                 Books.RemoveAll(a => a.BookId == bookId);
63:                 Books.Add(new BookRecord(book.BookId, book.BookAmount - amount,
book.Title));
64:             }
65:             else
66:             {
67:                 throw new ArgumentException($"Cannot return book with title
{book.Title.Value}. Amount to return is higher than {book.BookAmount.Amount}");
68:             }
69:
70:             AddEvent(new BookRemovedFromLibraryRecord(bookId, amount.Amount,
(LibraryRecordId)Id));
71:
72:             if (ReturnDate < DateTime.UtcNow)
73:             {
74:                 var oldFineValue = ReturnFine.Value;
75:                 var daysBetweenTodayAndReturnDate = ((DateTime.UtcNow -
ReturnDate)).Days;
76:                 var newFineValue = daysBetweenTodayAndReturnDate * 10;
77:                 ReturnFine = new ReturnFine(oldFineValue + newFineValue);
78:             }
79:
80:             if (ReturnFine.Value == 0)
81:             {
82:                 IsClosed = true;
83:             }
84:         }
```

```
85:
86:         public void PayFine()
87:         {
88:             if (Books.Count == 0)
89:             {
90:                 this.IsClosed = true;
91:             }
92:         }
93:
94:         public override void CheckState()
95:         {
96:             if (Books == null ||
97:                 User == null ||
98:                 User.IsBanned)
99:             {
100:                 throw new InvalidEntityStateException();
101:             }
102:         }
103:
104:         public void UpdateBookTitle(string bookId, string newTitle)
105:         {
106:             var book = Books.Single(a => a.BookId.Value == bookId);
107:             Books.Add(new BookRecord(book.BookId, book.BookAmount, new
BookTitle(newTitle)));
108:             Books.Remove(book);
109:         }
110:     }
111: }
```

4.1.2 Library.ApplicationLayer

Library application layer is a layer between client and Framework. It is responsible for providing interfaces for client or managing *data transfer objects*[8] from the client to the database and vice versa.

Static class `ApplicationServicesCollectionExtensions` provides service registration such as application facades[9] and register all Framework services with `AddMongoDbInfrastructure` method. To enable typed domain identifiers, they shall be registered as well. (see Code 4-6).

---

[8] An object that carries data between processes in order to reduce the number of method calls. (Fowler, 2003)

[9] Façade - Defines a simple interface for a complex subsystem (by referring to many different interfaces in the subsystem). (w3sDesign, 2014)

To set up the Framework successfully, Mongo database engine shall have at least one replica set. Example of MongoDB configuration file can be found in chapter 4.1.3.

Code 4-6

```
14: namespace Infrastructure.ApplicationLayer
15: {
16:     public static class ApplicationServicesCollectionExtensions
17:     {
18:         public static IServiceCollection AddApplicationLayerServices(this
IServiceCollection services, string databaseName)
19:         {
20:             services.AddTransient<IBookFacade, BookFacade>();
21:             services.AddTransient<IAuthorFacade, AuthorFacade>();
22:             services.AddTransient<IUserFacade, UserFacade>();
23:             services.AddTransient<ILibraryRecordFacade, LibraryRecordFacade>();
24:
25:             var settings = new
MongoDbSettings("mongodb://localhost:27017?connect=replicaSet", databaseName);
26:             services.AddMongoDbInfrastructure(settings);
27:
28:             BsonClassMap.RegisterClassMap(new BsonClassMap(typeof(AuthorId)));
29:             BsonClassMap.RegisterClassMap(new BsonClassMap(typeof(BookId)));
30:             BsonClassMap.RegisterClassMap(new BsonClassMap(typeof(LibraryRecordId)));
31:             BsonClassMap.RegisterClassMap(new BsonClassMap(typeof(UserId)));
32:
33:             return services;
34:         }
35:
36:     }
37: }
```

`ApplicationLayer` project contain event handlers for every type of event in **Library.Domain**. As an example, event handler for `BookCreated` event is shown below (see Code 4-7). This event handler updates author record and adds the book which was created to the list of author books.

Code 4-7

```
10: namespace Library.ApplicationLayer
11: {
12:     public class BookCreatedEventHandler : IEventHandler<BookCreated>
13:     {
14:         private readonly ILogger<BookCreatedEventHandler> logger;
15:         private readonly IRepository<Author, string> authorRepository;
16:         private readonly IRepository<Book, string> bookRepository;
```

```
17:
18:         public BookCreatedEventHandler(ILogger<BookCreatedEventHandler> logger,
IRepository<Author, string> authorRepository, IRepository<Book, string> bookRepository)
19:         {
20:             this.logger = logger;
21:             this.authorRepository = authorRepository;
22:             this.bookRepository = bookRepository;
23:         }
24:
25:         public async Task Handle(BookCreated @event)
26:         {
27:             var book = await bookRepository.GetByIdAsync(@event.BookId);
28:
29:             await authorRepository.ModifyAsync(author => {
30:                 var bookList = author.Books;
31:                 bookList.Add(new AuthorBookRecord(@event.BookId, book.Title));
32:                 author.UpdateBooks(bookList);
33:             }, @event.AuthorId);
34:             logger.LogInformation($"Books by author with id {@event.AuthorId} were
updated");
35:         }
36:     }
37: }
```

`ApplicationLayer` project also contain query objects. All these objects inherit from Query base class defined in **Infrastructure.MongoDB** and described in chapter 3.2.2. All query objects used in sample library application look very similar. An example of a query object which reads all opened library records is included below (see Code 4-8).

Code 4-8

```
12: namespace Library.ApplicationLayer.Query
13: {
14:     public class ValidLibraryRecordDetailsQuery : Query<LibraryRecordDetailDTO>
15:     {
16:         public ValidLibraryRecordDetailsQuery(IMongoDbContext dbContext) :
base(dbContext)
17:         {
18:         }
19:
20:         public override async Task<IList<LibraryRecordDetailDTO>> GetResultsAsync()
21:         {
22:             var records = base.dbContext.GetCollection<LibraryRecord>()
23:                 .AsQueryable()
24:                 .Where(a => !a.IsClosed)
```

```
25:                 .OrderByDescending(a => a.CreatedDate);
26:
27:             var recordDetails = new List<LibraryRecordDetailDTO>();
28:
29:             foreach (var item in records)
30:             {
31:                 recordDetails.Add(LibraryRecordMapper.MapTo(item));
32:             }
33:
34:             return recordDetails;
35:         }
36:     }
37: }
```

4.1.3 Library.Client

In order to start the application, MongoDB v4.2.3 and .NET Core 3.0 need to be installed and at least one replica set should be registered in `mongod.conf` configuration file (see mongod.conf example Code 4-9).

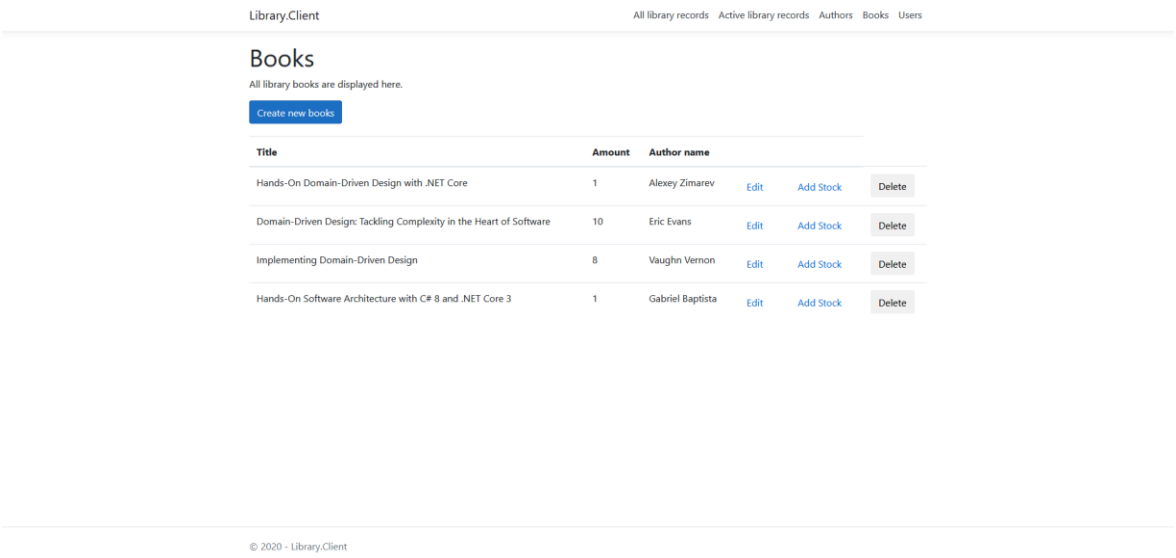
Code 4-9

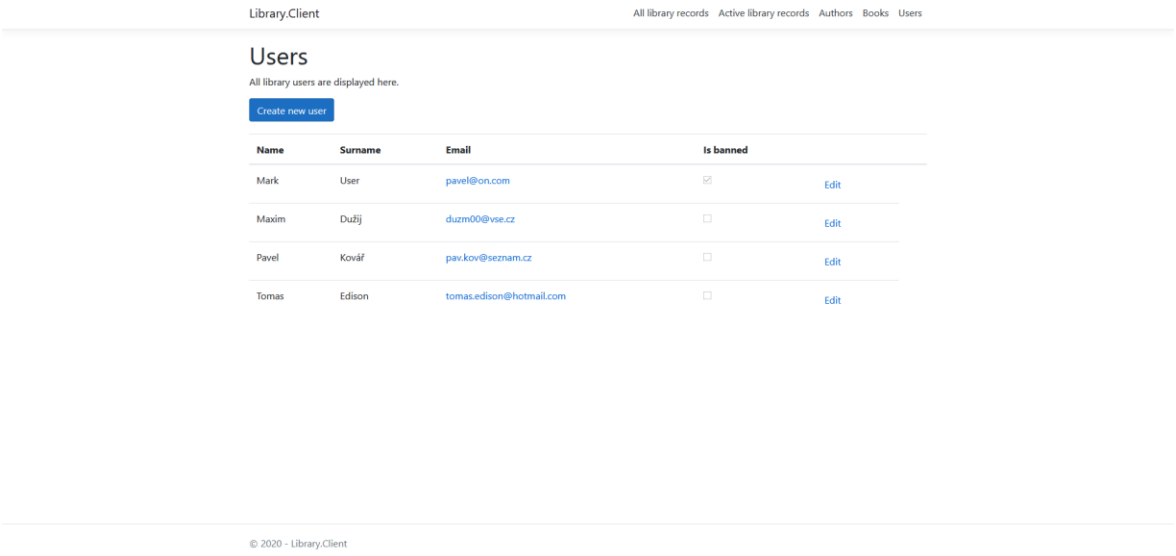
```
# mongod.conf

    storage:
      dbPath: C:\Program Files\MongoDB\Server\4.2\data
      journal:
        enabled: true

    systemLog:
      destination: file
      logAppend: true
      path:  C:\Program Files\MongoDB\Server\4.2\log\mongod.log

    net:
      port: 27017
      bindIp: 127.0.0.1

    replication:
      replSetName: rs0
      oplogSizeMB: 100
```

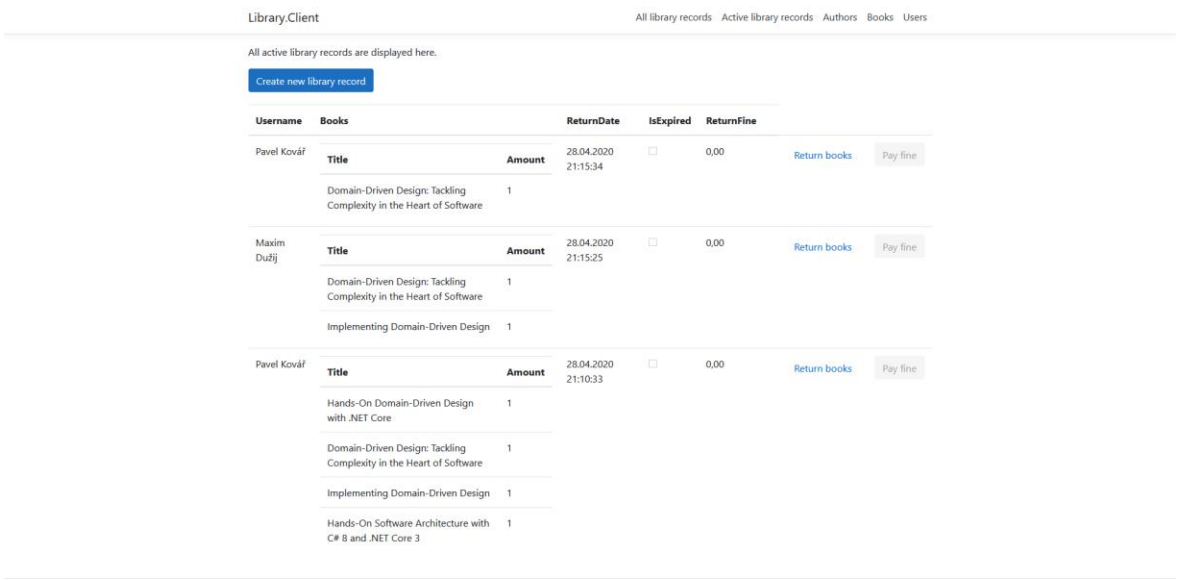
Library client is a simple Razor Pages web application with just 14 pages in total. Each page has a model where individual façade methods are called. This project has only one

dependency – **Library.ApplicationLayer**. Communication with the application layer is possible with the use of only data transfer objects.

Below, some application screenshots are shown.






Library.Client
All library records
Active library records
Authors
Books
Users
All active library records are displayed here.
Create new library record
Username
Books
ReturnDate
IsExpired
ReturnFine
Pavel Kovář
Title
Amount
Domain-Driven Design: Tackling Complexity in the Heart of Software
1
28.04.2020 21:15:34
0,00
Return books
Pay fine
Maxim Dužij
Title
Amount
Domain-Driven Design: Tackling Complexity in the Heart of Software
1
Implementing Domain-Driven Design
1
28.04.2020 21:15:25
0,00
Return books
Pay fine
Pavel Kovář
Title
Amount
Hands-On Domain-Driven Design with .NET Core
1
Domain-Driven Design: Tackling Complexity in the Heart of Software
1
Implementing Domain-Driven Design
1
Hands-On Software Architecture with C# 8 and .NET Core 3
1
28.04.2020 21:10:33
0,00
Return books
Pay fine

5 Conclusions

This bachelor's thesis covered some basic theory about domain-driven design and has described written framework internals and sample application implementations.

Framework v1.0 has limited functionality, but it was enough to successfully build an application using the DDD approach. Future improvements could cover some inner tooling to MongoDB migrations handling or extension points for data transformation.

Separation to **Infrastructure.Core** and **Infrastructure.MongoDB** creates the opportunity to add other database engines to Framework and to use several database technologies in the same project.

6 References

Evans, E., 2003. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. s.l.:Addison Wesley.

Fowler, M., 2003. *Data Transfer Object*. [Online] Available at: https://martinfowler.com/eaaCatalog/dataTransferObject.html [Accessed 16 04 2020].

Fowler, M., 2020. *DDD_Aggreagate*. [Online] Available at: https://www.martinfowler.com/bliki/DDD_Aggregate.html [Accessed 3 4 2020].

Microsoft, 2017. *Reflection in .NET*. [Online] Available at: https://docs.microsoft.com/en-us/dotnet/framework/reflection-and-codedom/reflection
[Accessed 21 04 2020].

Microsoft, 2018. *Domain events: design and implementation*. [Online] Available at: https://docs.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/domain-events-design-implementation
[Accessed 10 04 2020].

Microsoft, 2018. *Handling Concurrency Conflicts*. [Online] Available at: https://docs.microsoft.com/en-us/ef/core/saving/concurrency
[Accessed 21 04 2020].

Microsoft, 2019. *SQL Server Transaction Locking and Row Versioning Guide*. [Online] Available at: https://docs.microsoft.com/en-us/sql/2014-toc/sql-server-transaction-locking-and-row-versioning-guide?view=sql-server-2014
[Accessed 29 3 2020].

Microsoft, 2020. *Dependency injection in ASP.NET Core*. [Online] Available at: https://docs.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection?view=aspnetcore-3.1
[Accessed 21 04 2020].

Microsoft, 2020. *Implement value objects*. [Online] Available at: https://docs.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/implement-value-objects
[Accessed 6 4 2020].

MongoDB, 2019. *Release Notes for MongoDB 4.2*. [Online] Available at: https://docs.mongodb.com/manual/release-notes/4.2/ [Accessed 17 04 2020].

w3sDesign, 2014. *GoF design patter reference*. [Online] Available at: http://w3sdesign.com/?gr=s05&ugr=struct [Accessed 15 04 2020].

Wikipedia, 2019. *Eventual consistency*. [Online] Available at: https://en.wikipedia.org/wiki/Eventual_consistency [Accessed 6 4 2020].

Wikipedia, 2020. *Universally unique identifier*. [Online] Available at: https://en.wikipedia.org/wiki/Universally_unique_identifier [Accessed 6 4 2020].

Zimarev, A., 2019. *Hands-On Domain-DrivenDesign with .NET Core*. Birmingham, UK: Packt Publishing Ltd..